



SoLong

And thanks for all the fish!

Resumen: Este proyecto es un pequeño juego en 2D. Está diseñado para hacerte trabajar con texturas y sprites y algunos otros elementos básicos de jugabilidad.

Versión: 5.0

Índice general

I.	Introducción	2
II.	Objetivos	3
III.	Instrucciones generales	4
IV.	Instrucciones sobre la IA	5
V.	Parte obligatoria	8
V.1.	Juego	10
V.2.	Gestión de gráficos	10
V.3.	Mapa	11
VI.	Requisitos del Readme	12
VII.	Parte extra	13
VIII.	Ejemplos	14
IX.	Entrega y evaluación	15

Capítulo I

Introducción

Ser desarrollador es genial si quieres crear tu propio juego.
Pero un buen juego necesita recursos gráficos buenos.
Para juegos 2D, debes buscar tiles, tilesets, sprites y sprite sheets.
Por suerte para ti, algunos artistas con talento están dispuestos a compartir su trabajo en plataformas como:
itch.io

Hagas lo que hagas, respeta el trabajo de otros.

Capítulo II

Objetivos

¡Llegó el momento de crear un proyecto de diseño gráfico básico!
`so_long` te ayudará a mejorar tus habilidades en estas áreas: ventanas, colores, eventos, texturas, etc.

Vas a usar la librería gráfica del campus `MiniLibX`. Esta librería se ha desarrollado internamente e incluye las herramientas básicas necesarias para abrir una ventana, crear imágenes y trabajar con eventos de teclado y ratón.

Los objetivos de este proyecto son similares a los de este primer año: rigor, uso de `C`, uso de algoritmos básicos, búsqueda de información, etc.

Capítulo III

Instrucciones generales

- El proyecto deberá estar escrito en C.
- El proyecto debe estar escrito siguiendo la Norma. Si tienes archivos o funciones adicionales, estas deberán estar incluidas en la verificación de la Norma y tendrás un 0 si hay algún error de norma en cualquiera de ellos.
- Las funciones no deben terminar de forma inesperada (segfault, bus error, double free, etc), excepto en el caso de comportamientos indefinidos. Si esto sucede, el proyecto será considerado no funcional y recibirás un 0 durante la evaluación.
- Toda la memoria asignada en la pila (heap) deberá liberarse adecuadamente cuando sea necesario. No se permitirán leaks de memoria.
- Si el enunciado lo requiere, se deberá entregar un `Makefile` que compilará tus archivos fuente a la salida requerida con las flags `-Wall`, `-Werror` y `-Wextra`. También se deberá utilizar `cc` y, por supuesto, el `Makefile` no debe hacer `relink`.
- El `Makefile` entregado debe contener al menos las normas `$(NAME)`, `all`, `clean`, `fclean` y `re`.
- Para entregar los bonus del proyecto se deberá incluir una regla `bonus` en el `Makefile`, en la que se añadirán todos los headers, librerías o funciones que estén prohibidas en la parte principal del proyecto. Los bonus deben estar en archivos distintos `× _bonus.{c/h}`. La parte obligatoria y los bonus se evalúan por separado.
- Si el proyecto permite el uso de la `libft`, se deberá copiar su fuente y sus `Makefile` asociados en un directorio `libft` con su correspondiente `Makefile`. El `Makefile` del proyecto debe compilar primero la librería utilizando su `Makefile`, y después compilar el proyecto.
- Es recomendable crear programas de prueba para el proyecto, aunque este trabajo **no será entregado ni evaluado**. Esto ofrece la oportunidad de verificar que el programa funciona correctamente durante las evaluaciones. Y sí, está permitido utilizar estas pruebas durante cualquier evaluación.
- Entrega el trabajo en el repositorio `Git` asignado. Solo el trabajo de tu repositorio `Git` será evaluado. Si el proyecto tiene que ser evaluado por Deepthought, la evaluación se realizará después de las evaluaciones personales. Si, durante la evaluación de Deepthought, se encuentra un error se, se interrumpirá su evaluación.

Capítulo IV

Instrucciones sobre la IA

● Contexto

Durante tu proceso de aprendizaje, la IA puede ayudarte con muchas tareas diferentes. Tómate el tiempo necesario para explorar las diversas capacidades de las herramientas de IA y cómo pueden apoyarte con tu trabajo. Sin embargo, siempre debes abordarlas con precaución y evaluar de forma crítica los resultados. Ya sea código, documentación, ideas o explicaciones técnicas, nunca podrás saber con total certeza si tu pregunta está bien formulada o si el contenido generado es el adecuado. Las personas que te rodean son tu recurso más valioso para ayudarte a evitar errores y puntos ciegos.

● Mensaje principal:

- 👉 Utiliza la IA para reducir las tareas repetitivas o tediosas.
- 👉 Desarrolla habilidades de prompting, ya sea para programación o para otros temas, que beneficiarán tu futura carrera.
- 👉 Aprende cómo funcionan los sistemas de IA para anticipar de forma eficiente y evitar los riesgos comunes, sesgos y problemas éticos.
- 👉 Sigue trabajando con tus compañeros para desarrollar tanto habilidades técnicas como habilidades transversales.
- 👉 Utiliza únicamente contenido generado por IA que entiendas completamente y del cual puedas responsabilizarte.

● Reglas para estudiantes:

- Debes tomarte el tiempo necesario para explorar las herramientas de IA y comprender cómo funcionan, para poder utilizarlas de manera ética y reducir los sesgos potenciales.
- Debes reflexionar sobre tu problema antes de dar instrucciones a la IA. Esto te ayuda a escribir preguntas, instrucciones o conjuntos de datos más claros, detallados y relevantes utilizando un vocabulario preciso.

- Debes desarrollar el hábito de revisar, cuestionar y probar sistemáticamente cualquier contenido generado por la IA.
- Debes buscar siempre la revisión de otras personas, no te limites a confiar en tu propia validación.

● Resultados de esta etapa:

- Desarrollar habilidades de prompting tanto generales como de ámbito específico.
- Aumentar tu productividad con un uso eficaz de las herramientas de IA.
- Seguir fortaleciendo el pensamiento computacional, la resolución de problemas, la adaptabilidad y la colaboración.

● Comentarios y ejemplos:

- Ten en cuenta que la IA puede no tener la respuesta correcta porque esa respuesta no esté ni siquiera en Internet. Además, si te da soluciones incorrectas, intenta no insistir y busca ayuda entre las personas que te rodean. Vas a ahorrarte tiempo y vas a sumar en comprensión.
- Vas a enfretarte con frecuencia a situaciones (como exámenes o evaluaciones) donde debes demostrar una comprensión real. Prepárate, sigue construyendo tanto tus habilidades técnicas como transversales.
- Explicar tu razonamiento y debatir con otras personas suele revelar lagunas en tu comprensión de un concepto. Prioriza el aprendizaje entre pares.
- Lo normal es que la herramienta de IA que utilices no conozca tu contexto específico (a menos que se lo indiques), así que te dará respuestas genéricas. Si buscas información más adecuada y más precisa en relación a tu entorno cercano, confía en el resto de estudiantes.
- Donde la IA tiende a generar la respuesta más probable, el resto de estudiantes puede proporcionar perspectivas alternativas y matices valiosos. Confía en la comunidad de 42 como un punto de control de calidad.

✓ Buenas prácticas:

Le pregunto a la IA: "¿Cómo pruebo una función de ordenación?" Me da algunas ideas. Las pruebo y reviso los resultados con otra persona. Refinamos el enfoque de manera conjunta.

✗ Mala práctica:

Le pido a la IA que escriba una función completa, la copio y la pego en mi proyecto. Durante la evaluación entre pares, no puedo explicar qué hace ni por qué. Pierdo credibilidad. Suspenso mi proyecto.

✓ Buenas prácticas:

Utilizo la IA para ayudarme a diseñar un parser. Luego, reviso la lógica con otra persona. Encontramos dos errores y lo reescribimos juntos: mejor, más limpio y comprendiendo al 100

✗ Mala práctica:

Dejo que Copilot genere mi código para una parte clave de mi proyecto. Compila, pero no puedo explicar cómo maneja los pipes. Durante la evaluación, no puedo justificarlo y suspendo mi proyecto.

Capítulo V

Parte obligatoria

Nombre de programa	so_long
Archivos a entregar	Makefile, *.h, *.c, maps, textures
Makefile	all, clean, fclean, re, bonus
Argumentos	un mapa en formato *.ber
Funciones autorizadas	• open, close, read, write, malloc, free, perror, strerror, exit • Todas las funciones de la librería math (flag del compilador -lm, man 3 math) • Todas las funciones de la miniLibX • ft_printf y cualquier función equivalente que tú hayas escrito
Se permite usar libft	Sí
Descripción	Debes crear un pequeño juego 2D donde un delfín escape del planeta Tierra después de comer pescado...En vez de un delfín, peces y la Tierra, puedes usar cualquier personaje, cualquier objeto y lugar que quieras.

Tu proyecto debe cumplir con las siguientes normas:

- Debes usar la miniLibX. Ya sea la versión disponible en los equipos del campus, o instalandola desde su fuente original.

- Debes entregar un **Makefile** que compile con tus archivos fuente. Este no debe hacer relink.
- Tu programa debe aceptar como primer argumento un archivo con la descripción del mapa de extensión **.ber**.

V.1. Juego

- El objetivo del jugador es recolectar todos los objetos presentes en el mapa y salir eligiendo la ruta más corta posible.
- Las teclas W, A, S y D se utilizarán para mover al personaje principal.
- El jugador debe poder moverse en 4 **direcciones**: subir, bajar, ir a la izquierda o ir a la derecha.
- El jugador no puede entrar dentro de las paredes.
- Tras cada movimiento, el **número real de movimientos** debe mostrarse en un terminal.
- Utilizarás una **perspectiva 2D** (vista de pájaro o lateral).
- El juego no necesita ser en tiempo real.
- Aunque los ejemplos dados se refieren a una temática de delfín, puedes crear el mundo que quieras.



Si lo prefieres, puedes utilizar ZQSD o las teclas de dirección para mover el personaje principal.

V.2. Gestión de gráficos

- El programa mostrará la imagen en una ventana.
- La gestión de tu ventana debe ser limpia (cambiar de ventana, minimizar, etc)
- Pulsar la tecla **ESC** debe cerrar la ventana y cerrar el programa limpiamente.
- Hacer clic en la cruz de la ventana debe cerrar la ventana y terminar el programa limpiamente.
- El uso de **images** de la **miniLibX** es obligatorio

V.3. Mapa

- El mapa estará construido de 3 componentes: **paredes**, **objetos** y **espacio abierto**.
- El mapa estará compuesto de solo 5 caracteres: **0** para un espacio vacío, **1** para un muro, **C** para un coleccionable, **E** para salir del mapa y **P** para la posición inicial del jugador.

Este es un ejemplo simple de un mapa válido:

```
1111111111111111
100100000000C1
1000011111001
1P0011E000001
1111111111111111
```

- El mapa debe tener una salida, al menos un objeto y una posición inicial.



Si el mapa contiene caracteres duplicados (salida o posición inicial), deberás mostrar un mensaje de error.

- El mapa debe ser rectangular.
- El mapa deberá estar cerrado/rodeado de muros, en caso contrario el programa deberá devolver un error.
- Tienes que comprobar si hay un camino válido en el mapa.
- Debes poder procesar cualquier tipo de mapa, siempre y cuando respete las anteriores normas.
- Otro ejemplo minimalista de un mapa **.ber**:

[illegible]

- En caso de fallos de configuración de cualquier tipo encontrados en el archivo, el programa debe terminar correctamente y devolver “Error\n” seguido de un mensaje explícito de tu elección.

Capítulo VI

Requisitos del Readme

Debe incluirse un archivo `README.md` en la raíz del repositorio Git. Su propósito es permitir que cualquier persona que no esté familiarizada con el proyecto (pares, personal, responsables de selección, etc.) pueda entender rápidamente de qué trata el proyecto, cómo ejecutarlo y dónde encontrar más información sobre el tema.

El `README.md` debe incluir, como mínimo:

- La primera línea debe estar en cursiva y decir: *Este proyecto ha sido creado como parte del currículo de 42 por <login1>[, <login2>[, <login3>[...]]]*.
 - Una sección de "**Descripción**" que presente claramente el proyecto, incluyendo su objetivo y una breve visión general.
 - Una sección de "**Instrucciones**" que contenga cualquier información relevante sobre compilación, instalación y/o ejecución.
 - Una sección de "**Recursos**" que enumere referencias clásicas relacionadas con el tema (documentación, artículos, tutoriales, etc.), así como una descripción del uso de IA, especificando para qué tareas y en qué partes del proyecto se ha utilizado.
- Podrían requerirse secciones adicionales dependiendo del proyecto (por ejemplo, ejemplos de uso, lista de características, decisiones técnicas, etc.).

Cualquier contenido extra requerida se listará explícitamente a continuación.



Es recomendable escribir el README en inglés; sin embargo, también puedes usar el idioma oficial de tu campus.

Capítulo VII

Parte extra

Normalmente te animaríamos a que desarrollaras algunas funcionalidades extras originales tuyas. Sin embargo, habrá proyectos gráficos mucho más interesantes más adelante. ¡Te están esperando! ¡No pierdas demasiado tiempo en este encargo!

Se permite el uso de otras funciones para completar la parte extra, siempre y cuando su uso **se justifique** durante la evaluación. Sé inteligente.

Conseguirás puntos extra si:

- Haces que el jugador pierda cuando toque una patrulla de enemigos
- Añades algunas animaciones de sprites.
- Muestras el contador de movimiento directamente en la pantalla en lugar de en el terminal.



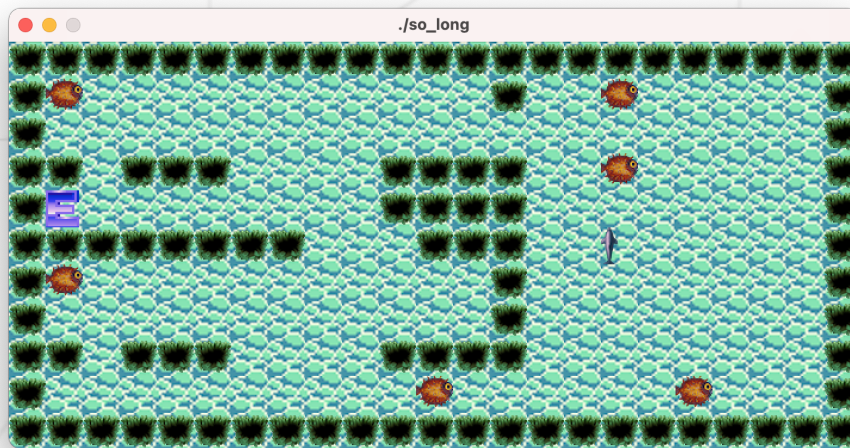
Puedes añadir tantos archivos o carpetas como necesiten tus bonus.



La parte bonus será evaluada si y sólo si la parte obligatoria está PERFECTA. Perfecta quiere decir que toda la parte obligatoria funciona correctamente y la gestión de errores es impecable. Si no has superado todos los requisitos de la parte obligatoria, el bonus no será evaluado en absoluto.

Capítulo VIII

Ejemplos



Algunos ejemplos de `so_long` con muy mal gusto en diseño gráfico.
(Pero casi valen algunos puntos de bonus!)

Capítulo IX

Entrega y evaluación

Entrega tu trabajo en tu repositorio `Git` como de costumbre. Solo el trabajo de tu repositorio será evaluado. Comprueba dos veces si es necesario que los nombres de tus archivos sean los correctos.

Como estos ejercicios no se verifican con un programa, puedes organizar tus archivos como quieras, siempre que entregues los archivos obligatorios y cumplas los requisitos.

Durante la evaluación, es posible que se solicite una ligera **modificación del proyecto**. Esto puede consistir en ajustar ligeramente el comportamiento, modificar unas cuantas líneas de código o incorporar una característica fácil de implementar.

Puede que este paso **no sea necesario en todos los proyectos**, pero hay que tenerlo en cuenta si así se especifica en la hoja de evaluación.

Este paso sirve para verificar la comprensión real de una parte específica del proyecto. La modificación se puede realizar en cualquier entorno de desarrollo que se elija (por ejemplo, la configuración habitual), y debería ser factible en unos pocos minutos, a menos que se defina un plazo específico como parte de la evaluación.

Por ejemplo, se puede pedir hacer una pequeña actualización en una función o *script*, modificar lo que se vería en pantalla o ajustar una estructura de datos para almacenar nueva información, etc.

Los detalles (alcance, objetivo, etc.) se especificarán cada **hoja de evaluación** y pueden variar de una evaluación a otra para el mismo proyecto.